In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:
```
df = pd.read_csv("project_shopping_trends.csv")
```

In [3]:
```
df.head()
```

Out[3]:

	Customer_ID	Age	Gender	Item_Purchased	Category	Purchase_Amount_USD	Loc
0	Customer_ID	Age	Gender	Item_Purchased	Category	Purchase_Amount_USD	Loc
1	1	55	Male	Blouse	Clothing	53	Ken
2	2	19	Male	Sweater	Clothing	64	N
3	3	50	Male	Jeans	Clothing	73	Massachu
4	4	21	Male	Sandals	Footwear	90	Rhode I

In [4]:
```
df.columns
```

Out[4]:
```
Index(['Customer_ID', 'Age', 'Gender', 'Item_Purchased', 'Category',
       'Purchase_Amount_USD', 'Location', 'Size', 'Color', 'Season',
       'Review_Rating', 'Subscription_Status', 'Payment_Method',
       'Shipping_Type', 'Discount_Applied', 'Promo_Code_Used',
       'Previous_Purchases', 'Preferred_Payment_Method',
       'Frequency_of_Purchases'],
      dtype='object')
```

In [5]:
```
df.columns = (
    df.columns.str.strip()
    .str.lower()
    .str.replace(" ", "_")
    .str.replace("(", "")
    .str.replace(")", "")
)
```

In [6]:
```
df.columns
```

Out[6]:
```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount_usd', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'payment_method',
       'shipping_type', 'discount_applied', 'promo_code_used',
       'previous_purchases', 'preferred_payment_method',
       'frequency_of_purchases'],
      dtype='object')
```

In [8]:
```
df.head()
```

Out[8]:

	customer_id	age	gender	item_purchased	category	purchase_amount_usd	locati
0	Customer_ID	Age	Gender	Item_Purchased	Category	Purchase_Amount_USD	Locati
1	1	55	Male	Blouse	Clothing	53	Kentuc
2	2	19	Male	Sweater	Clothing	64	Mai
3	3	50	Male	Jeans	Clothing	73	Massachuse
4	4	21	Male	Sandals	Footwear	90	Rhode Isla

In [14]:

```
# Auto-detect category column
cat_col = [c for c in df.columns if "cat" in c.lower()][0]

# Auto-detect purchase amount column
amt_col = [c for c in df.columns if "purchase" in c.lower()][0]

# FORCE purchase amount to numeric (THIS FIXES agg error)
df[amt_col] = pd.to_numeric(df[amt_col], errors="coerce")

# Now aggregation will work
df.groupby(cat_col)[amt_col].mean()
```

Out[14]:

```
category
Accessories   NaN
Category      NaN
Clothing      NaN
Electronics   NaN
Footwear      NaN
Home Decor    NaN
Outerwear     NaN
Sports        NaN
Name: item_purchased, dtype: float64
```

In [16]:

```
# Auto-detect age column
age_col = [c for c in df.columns if "age" in c.lower()][0]

# Force age column to numeric
df[age_col] = pd.to_numeric(df[age_col], errors="coerce")

# Create age groups safely
df["age_group"] = pd.cut(
    df[age_col],
    bins=[0, 30, 45, 100],
    labels=["18-30", "31-45", "46+"]
)

# Age group spending
df.groupby("age_group")[amt_col].mean()
```

```
C:\Users\hp\AppData\Local\Temp\ipykernel_22892\546892921.py:15: FutureWarning: The d
efault of observed=False is deprecated and will be changed to True in a future versi
on of pandas. Pass observed=False to retain current behavior or observed=True to ado
pt the future default and silence this warning.
  df.groupby("age_group")[amt_col].mean()
```

Out[16]:

```
age_group
18-30   NaN
31-45   NaN
46+     NaN
Name: item_purchased, dtype: float64
```

In [18]:

```
# Auto-detect season column
season_col = [c for c in df.columns if "season" in c.lower()][0]

# Make sure purchase amount is numeric (safe re-check)
df[amt_col] = pd.to_numeric(df[amt_col], errors="coerce")

# Season-wise spending
df.groupby(season_col)[amt_col].mean()
```

Out[18]:

```
season
Fall     NaN
Season   NaN
Spring   NaN
Summer   NaN
Winter   NaN
Name: item_purchased, dtype: float64
```

In [19]:

```
cluster_df = df[
    [
        "age",
        "purchase_amount_usd",
        "previous_purchases",
        "review_rating",
        "frequency_of_purchases",
        "discount_applied",
        "subscription_status"
    ]
].copy()
```

In [20]:

```
cluster_df.dtypes
```

Out[20]:

```
age                       float64
purchase_amount_usd        object
previous_purchases         object
review_rating              object
frequency_of_purchases     object
discount_applied           object
subscription_status        object
dtype: object
```

In [21]:

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
```

```
for col in cluster_df.columns:
    if cluster_df[col].dtype == "object":
        cluster_df[col] = le.fit_transform(cluster_df[col].astype(str))
```

In [22]:
```
cluster_df.dtypes
```

Out[22]:
```
age                       float64
purchase_amount_usd         int64
previous_purchases          int64
review_rating               int64
frequency_of_purchases      int64
discount_applied            int64
subscription_status         int64
dtype: object
```

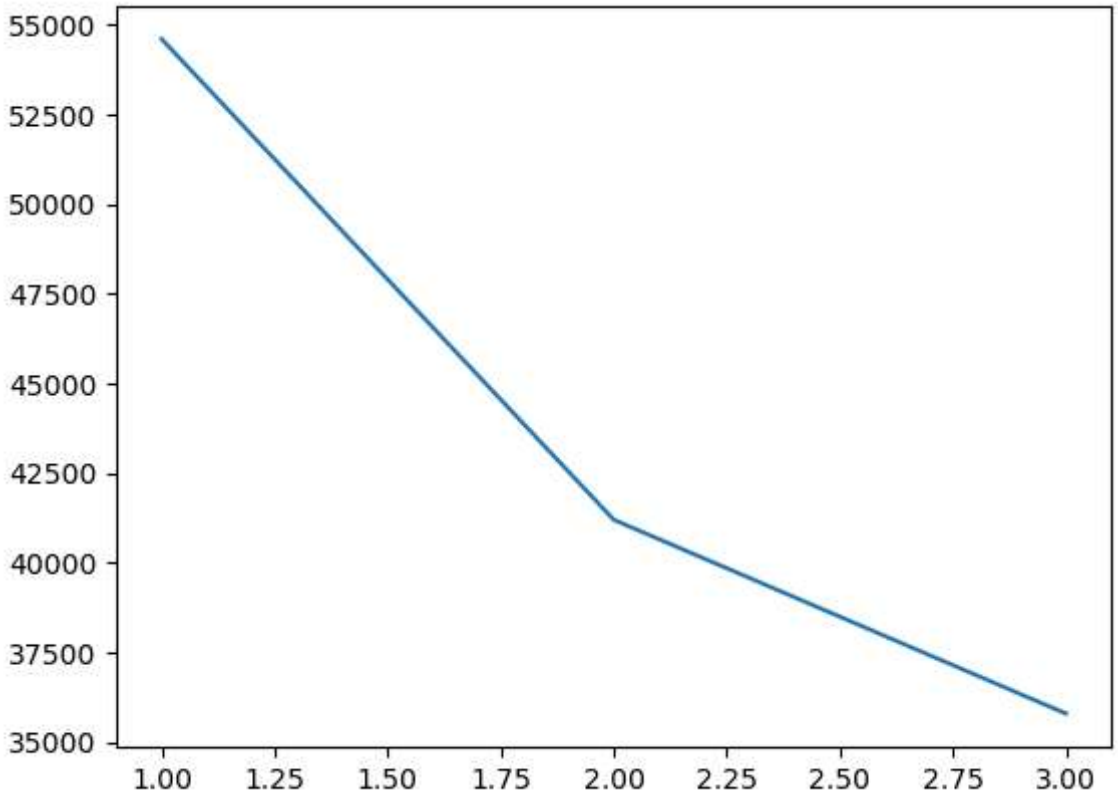
In [23]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled_data = scaler.fit_transform(cluster_df)
```

In [26]:
```
# Remove rows with missing values (SAFE)
cluster_df = cluster_df.dropna()

# Re-scale after dropping NaN
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaled_data = scaler.fit_transform(cluster_df)
```

In [27]:
```
print(scaled_data.shape)
```

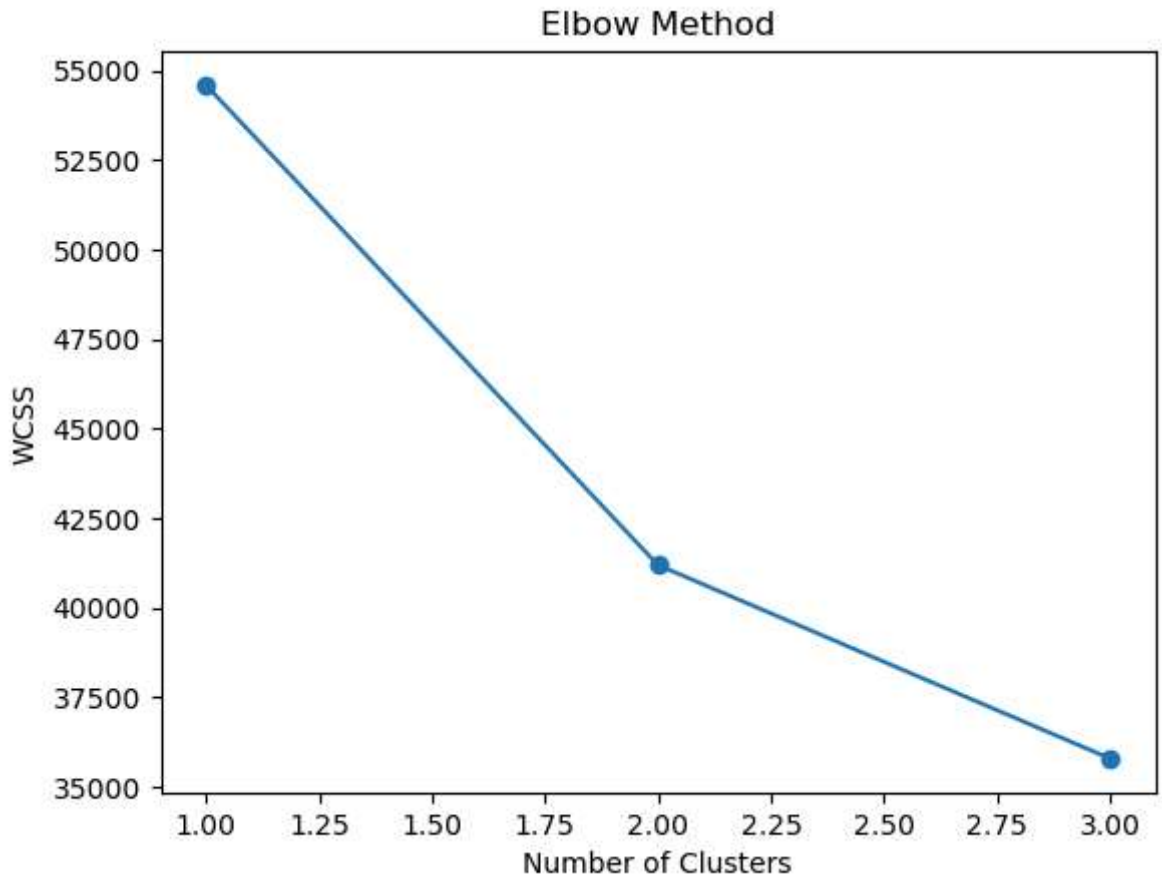
```
(7800, 7)
```

In [29]:
```
wcss = []
for i in range(1, 4):   # reduced clusters
    kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)
    kmeans.fit(scaled_data)
    wcss.append(kmeans.inertia_)

plt.plot(range(1,4), wcss)
plt.show()
```



In [30]:
```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Safety check
print("Shape:", scaled_data.shape)

wcss = []
for i in range(1, 4):
    kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)
    kmeans.fit(scaled_data)
    wcss.append(kmeans.inertia_)

plt.plot(range(1,4), wcss, marker="o")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```

```
Shape: (7800, 7)
```



In [32]:
```
df = df.loc[cluster_df.index].copy()
```

In [33]:
```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df["cluster"] = kmeans.fit_predict(scaled_data)
```

In [34]:
```
df["cluster"].value_counts()
```

Out[34]:
```
cluster
0    3730
2    2326
1    1744
Name: count, dtype: int64
```

In [35]:
```
df.groupby("cluster").mean(numeric_only=True)
```

Out[35]:

	age	item_purchased
cluster		
0	41.844236	NaN
1	43.254587	NaN
2	43.433792	NaN

In [36]:
```
df.to_csv("final_clustered_data.csv", index=False)
```

In [ ]: